Chapter 1. Introduction

GenAI is so powerful and easy to use that even nontechnical users can easily prototype very compelling applications on top of GenAI. However, taking such GenAI prototypes to production is hard because GenAI models are unreliable—they can hallucinate, return different answers to the same input, and can have surprising limitations because of how they are trained. The design patterns in this book capture best practices and solutions to these and other recurring problems you're likely to encounter when building production applications on top of GenAI models.

GenAI Design Patterns

Design patterns, in software engineering, are proven solutions to common problems that occur during software design and development. They represent standardized best practices that have evolved over time through the collective experience of software developers. Design patterns are important because they establish a common vocabulary developers can use to communicate efficiently and because they help improve software quality, maintainability, and scalability.

Preface
1. Introduction
Generative AI Design Patterns
2. Controlling Content Style

cant prominence in software engineering with the publication of the book *Design Patterns: Elements of Reusable Object-Oriented Software* by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (Addison-Wesley), which is often called "the Gang of Four book." Since then, design patterns have been cataloged for other software engineering domains, such as for Java Enterprise applications and ML.

When building AI products today, developers increasingly turn to *foundational* GenAI models (such as GPT-4, Gemini, Claude, Llama, DeepSeek, Qwen, and Mistral) that are trained on large, application-agnostic datasets, rather than building custom ML models that need to be trained from scratch on application-specific data. In this book, we'll follow Chip Huyen's *AI Engineering* (O'Reilly) in referring to this approach of building on top of foundational models as *AI engineering* and to practitioners of this approach as *AI engineers*.

AI engineering has a wide range of applications—including natural-language processing (NLP), text generation, code explanation, image understanding, and video synthesis—to power use cases such as content generation, AI assistants, workflow automation, and robotics.

As an AI engineer, you can ask a foundational model to directly generate the content your application needs by sending the model an appropriate text input, which is known as a *prompt*. However, you will face certain common problems—the generated content may not match the style you want, may be missing enterprise knowledge that the model doesn't know about, or may lack certain capabilities. In this book, we catalog a variety of proven solutions to such problems that arise in the context of building applications on top of GenAI foundational models.

In this book, you will also find detailed explanations of 32 patterns that codify research advances and the experience of experts into advice that you can readily incorporate into your projects. Each chapter offers a set of patterns as potential solutions to a particular problem that commonly arises in AI engineering. For example, Chapter 3 is about solving the problem that foundational models can't generate content that is informed by confidential enterprise data, because they are trained by model providers who don't have access to that data. The patterns presented in that chapter all address this problem. Each section that presents a pattern includes a description of the problem, a proven solution, an end-to-end working example of the pattern, and a discussion of alternatives and other considerations for implementing it.

AI engineers often encounter tasks that are too complex for a foundational model to perform all at once, so a common tactic is to break the complex task into smaller components that can be accomplished by foundational models. Such small software components that provide capabilities with the help of foundational models are called *agents*. Agents become increasingly autonomous as they use GenAI models to plan out a sequence of operations, identify the backend tools that they can invoke for each operation, determine how to recover from errors, and/or evaluate whether the task is complete. Applications that are built by orchestrating agents are called *agentic*. By showing you how to handle the inevitable challenges that arise when building applications on foundational models, the patterns in this book will help you build better agents and agentic applications.

Building on Foundational Models

In this section, we'll quickly cover the basics of AI engineering so that we don't have to repeat this introductory material in the sections on the patterns that follow in later chapters. For deeper coverage of building GenAI applications, we refer you to books such as Omar Sanseviero et al.'s *Hands-On Generative AI with Transformers and Diffusion Models* (O'Reilly), which covers the underlying technology; Chris Fregly et al.'s *Generative AI on AWS* (O'Reilly), which covers hyperscaler offerings; and Leonid Kuligin et al.'s *Generative AI on Google Cloud with LangChain* (Packt), which covers an open source GenAI framework.

A NOTE ON MODELS AND FRAMEWORKS

In *Machine Learning Design Patterns*, we used just two frameworks (scikit-learn and TensorFlow) and a single hyperscaler (Google Cloud Platform [GCP]) for consistency, but many readers felt that the resulting examples were too TensorFlow- and GCP-heavy. Therefore, in this book, we endeavor to be agnostic to model, framework, and hyperscaler.

Our code examples employ a wide range of technologies from a number of different vendors: large language models (LLMs) from OpenAI, Anthropic, Google, Alibaba, and Meta; and GenAI frameworks like LangChain, Pydantic AI, Hugging Face, and DSPy. Our examples are also agnostic to hyperscalers such as Amazon Web Services (AWS), Azure, GCP, and Oracle Cloud Infrastructure. Since you're likely to be using a different model in a different framework to address a different scenario, the code examples are meant only to serve as starting points for your implementation—we fully intend that you will have to adapt the code examples to your preferred LLM, framework, and hyperscaler.

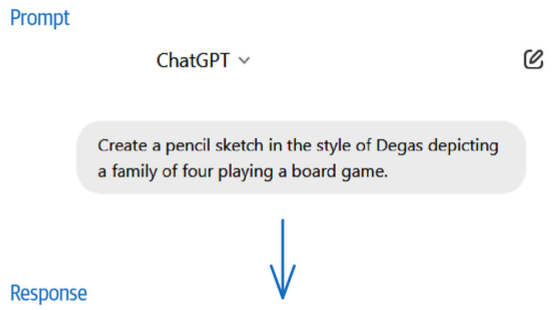
Prompt and Context

When you build AI applications, you typically invoke hosted foundational models through an API. This might be the API provided by the vendor of the foundational model, or it might be a framework that allows you to easily switch between providers.

You invoke a foundational model by sending it a *prompt* and getting back a *response*. You are, doubtless, familiar with doing this by using the web user interface of a foundational model. For example, on ChatGPT, you might type a prompt like this one into the text box:

> Create a pencil sketch in the style of Degas depicting a family of four playing a board game

The simplest prompt typically consists of an *instruction* to the model that asks it to perform some content-generation task. In this case, the model follows the instruction and sends back a response that contains an image of the type requested (see Figure 1-1).[1] Both prompts and responses can be *multimodal*—they could be text, but they could also be images, video, or audio.



Create a pencil sketch in the style of Degas depicting a family of four playing a board game.

Figure 1-1. Sending a prompt to a foundational model to get back a response

A more complex prompt might consist not just of an instruction but also *context*. This might be information that the model should use in the generation, or it might involve asking the model to take on a specific role. For example, we could tell Claude (on Claude.ai) to take on a personality and use some information when generating content:

> You are an expert marketer who is very familiar with the book market in university towns in Germany.
>
> *Covenant of Water* is a novel that tells the story of three generations of an Orthodox Saint Thomas Christian family in Kerala.
>
> Write a one-paragraph blurb introducing the book to readers at a bookstore in Göttingen, drawing local connections.

The response from Claude Sonnet takes into account the role, context, and instruction:

> **Covenant of Water - A Literary Journey to Kerala**
>
> Nestled between the historic walls of Göttingen, where academic tradition meets literary curiosity, we present Abraham Verghese's masterful epic, "The Covenant of Water." This sweeping saga follows three generations of an Orthodox Saint Thomas Christian family in Kerala, India—a region whose complex religious heritage might resonate with students of our university's renowned theological studies program. As the story unfolds across decades amid Kerala's lush backwaters, readers familiar with Göttingen's own relationship with water—from the sacred Gänseliesel fountain to the meandering Leine River—will find surprising parallels in how geography shapes destiny. Verghese weaves a tale of family secrets, medical mysteries, and generational bonds that transcends cultural boundaries, offering our community of international scholars and lifelong learners a profound exploration of how water—whether in Kerala's monsoons or Lower Saxony's gentle rains—becomes both metaphor and covenant, connecting us across time and space.

Using the Model Provider's API

APIs that are designed to invoke foundational models are available in a wide variety of languages—Python, Go, Java, and TypeScript are all commonly employed. For simplicity, we focus on Python APIs in this book.

To invoke Claude Sonnet 3 through the Anthropic API in Python, you could use the Anthropic Python package as follows (the full code is on GitHub):

```
import anthropic
client = anthropic.Anthropic(
    # defaults to os.environ.get("ANTHROPIC_API_KEY")
    api_key="YOUR_ANTHROPIC_API_KEY",
)

completion = client.messages.create(
    model="claude-3-7-sonnet-latest",
    system="You are an expert Python programmer.",
```

```
    messages=[
        {
            "role": "user",
            "content": [
               {
                  "type": "text",
                  "text": """
Write code to find the median value of a list of integers.
"""
               }
            ]
        }
    ]
)

print(completion.content[0].text)
```

In this API call, the prompt has been broken into two parts—a *system prompt* and a *user prompt*. The system prompt is set by the developer and guides the model's overall behavior, while the user prompt is more dynamic and provides specific instructions for a specific task you want the model to perform. Here, the AI assistant's role has been set in the system prompt while the user prompt and context are sent as messages.

Using an LLM-Agnostic Framework

To perform the same task using the PydanticAI framework, you'd use code such as the following (assuming the needed API key is set in an environment variable):

```
from pydantic_ai import Agent
agent = Agent('anthropic:claude-3-7-sonnet-latest',
              system_prompt="You are an expert Python programme

result = agent.run_sync(
           "Write code to find the median value of a list of in
print(result.data)
```

The advantage here is that you can easily switch between foundational model providers by switching the model string to `openai:gpt-4o-mini`, `google-vertex:gemini-2.0-flash`, `groq:llama3-70b-8192`, and so on (see Pydantic's documentation for the full list of models supported).

The class in the Pydantic API that invokes the Claude model is called `Agent`. We'll discuss what agents are in the next section, but before that, let's conclude our discussion of ways to invoke foundational models.

Running Your Model Locally

To run a model such as Llama 3 on your local hardware, you could use the Ollama client to download and run the model that you want to use:

```
ollama run llama3.2
```

Ollama exposes open-weights models with the OpenAI API, so you could use this:

```
from pydantic_ai.models.openai import OpenAIModel
from pydantic_ai.providers.openai import OpenAIProvider

model = OpenAIModel(
    model_name='llama3.2',
    provider=OpenAIProvider(base_url='http://localhost:11434/v1
)
```

How Foundational Models Are Created

Unlike traditional machine learning applications, your AI applications will rarely include a training phase. Instead, you'll build them on top of general-purpose foundational models that have been pretrained to perform a wide variety of tasks. You can, for the most part, ignore the internal details of the foundational model—and we're including this section in this book only so you can understand the associated vocabulary.

At the time of writing in spring 2025, DeepSeek is the foundational model with the most available information on its training regimen. We'll use that information to discuss the key steps (see Figure 1-2) involved in creating a foundational model. While OpenAI, Google, Anthropic, and Meta may not have followed this exact process in creating GPT, Gemini, Claude, and Llama, their methods are probably broadly similar.

The DeepSeek base LLM was pretrained (in Step 1 of Figure 1-2) on a diverse, high-quality corpus of 14.8 trillion tokens. (The works of Shakespeare, as a point of comparison, amount to about 1.2 million tokens—so the DeepSeek training dataset is equivalent to 12 million Shakespeares!) Unlike early LLMs, which were trained on words, modern LLMs are trained on *tokens*, which are short sequences of characters. This allows such LLMs to learn things that are not in the vocabulary of the language, like proper names. It isn't just size that helps—the DeepSeek team attributes the high quality of their models to careful data curation, including rigorous deduplication processes.

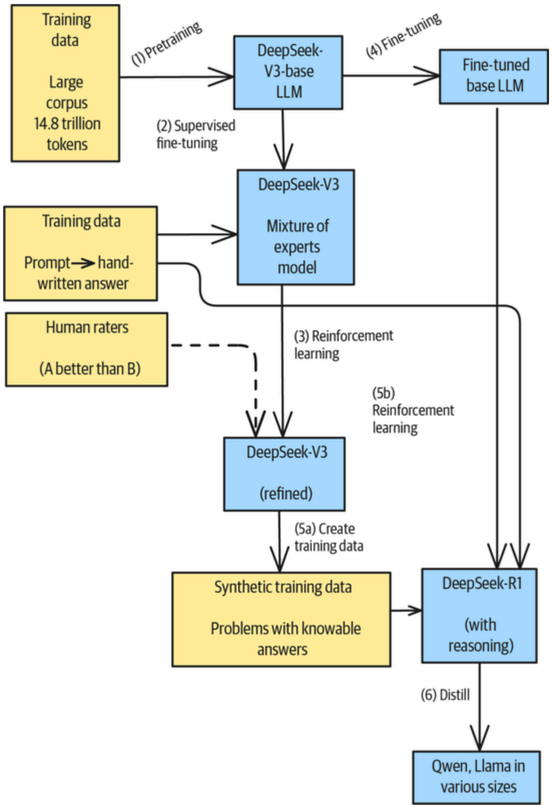


Figure 1-2. Stages involved in training the DeepSeek-R1 model

The *pretraining* stage (Step 1 in Figure 1-2) involves training on this massive dataset of tokens to develop the model's general language-understanding capabilities. The key goal at this stage is to train the model to predict the next token, given a context consisting of the previous tokens in the training input. This is why you'll often hear people call LLMs *next-token predictors*—but next-token prediction is only the first stage of the training regimen.

Following pretraining, the model undergoes *supervised fine-tuning* (SFT, Step 2 in Figure 1-2) to improve its ability to follow instructions and generate high-quality responses. This stage uses carefully curated datasets of human-written examples. Cohere, for example, has said it uses licensed physicians, financial analysts, and accountants to improve its models. Presumably, these practitioners write ideal answers given a prompt. The result, DeepSeek-V3, is a *mixture of experts* (MoE) model: an optimization that allows models to have a large number of parameters while activating only a fraction of them for each token. DeepSeek-V3 has 671 billion parameters in total, but only 37B are activated per token. This allows the model to use different pathways for different types of instructions.

The *reinforcement learning* stage (Step 3 in Figure 1-2) is where the models are further refined based on human preferences. This step involves *reinforcement learning with human feedback* (RLHF), which means showing human raters a pair of generated outputs and asking them which one they prefer. Such *preference tuning* helps align the models' outputs with human expectations and values. This stage is also sometimes called *preference optimization*.

Once DeepSeek-V3 was created with a small number of human-written examples and human preferences, the full DeepSeek-R1 model was developed through the following multistage process:

1. The *cold start* involved fine-tuning a base model (DeepSeek-V3-Base) with thousands of cold-start data points to lay a foundation.
2. *Pure reinforcement learning* (RL) involved applying a pure reinforcement learning approach to enhance reasoning skills.
3. *Rejection sampling* involved choosing the best examples from a distribution of data created during the last successful RL run.
4. SFT involved merging the synthetic data with supervised data from DeepSeek-V3-Base in domains like writing, factual question answering (QA), and self-cognition.
5. *Final RL* was a final reinforcement learning process across diverse prompts and scenarios.

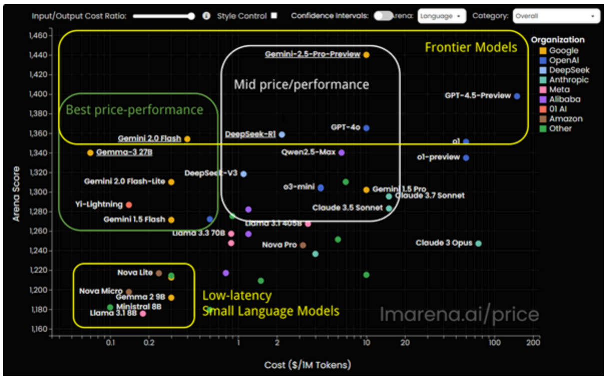
Step 2 (pure RL) was a major breakthrough because the team directly applied reinforcement learning to the base model (DeepSeek-V3-Base) *without* relying on supervised fine-tuning as a preliminary step. This approach allowed the model to explore *chain-of-thought* (CoT) reasoning for solving complex problems, which resulted in the development of DeepSeek-R1-Zero. It turned out that reasoning capabilities in LLMs can be incentivized purely through RL, without the need for SFT, as researchers had long believed. Since human-written examples are expensive, the ability to use pure RL allows for longer training on a much more diverse set of problems than if SFT is required.

To make DeepSeek-R1's capabilities more accessible, the team created distilled versions of the model that can run on more modest hardware while retaining much of the original's reasoning capability. These include models based on Qwen and Llama architectures in various sizes (1.5B, 7B, 8B, 14B, etc.).

The Landscape of Foundational Models

The GenAI foundational model ecosystem has evolved significantly, with distinct categories emerging to serve different needs.

Because academic benchmarks are saturated and can be gamed, the currently most accepted way to rate GenAI models is to compare them pair-wise in blind tests. LMArena carries out a large-scale comparison, and Figure 1-3 shows the April 2025 leaderboard.



Arena: Language
Category: Overall

Figure 1-3. The LMArena leaderboard on April 6, 2025 (annotated boxes added by the authors)

The leaderboard shows model rating on the y-axis and cost on the x-axis. You can use this to determine the best model that fits your budget and the lowest price you should expect to pay for a given rating. Note that the Elo rating on the y-axis is a logarithmic relationship,[2] and so is the cost on the x-axis.

The highest-rated models overall (see the y-axis) on the day we screenshotted the leaderboard were Gemini 2.5 Pro Preview, GPT-4.5 (Preview), GPT-4o, and DeepSeek-R1, with Gemini 2.0 Flash and OpenAI o1 also nearly as high. The ranking changes daily as new models are released and more ratings are collected, but the leaderboard has been remarkably consistent over time, with the flagship models from Google, OpenAI, and Anthropic almost always on top. Together, these are referred to as *frontier models*.

Frontier models such as GPT 5 and Gemini 2.5 Pro represent the state of the art in language model capabilities, and they offer the highest performance across reasoning, knowledge, and multimodal tasks. However, they are resource intensive and costly and can't be run locally due to their size and proprietary nature. You'd use them in enterprise-grade applications requiring sophisticated reasoning and where speed is not a concern. Recent developments in frontier models include multimodal capabilities, enhanced reasoning, and extended-content windows (with up to two million tokens in some models).

Distilled versions of frontier models balance performance with efficiency, offering reasonable capabilities at lower costs and with faster response times. Leading examples include Gemini Flash, Claude Sonnet, and GPT-4o-mini. These models tend to offer good performance on common tasks like content generation and summarization. They also offer fast response times, and they're cost-effective for high-volume applications. The cost difference between running Gemini Pro and Gemini Flash becomes quite apparent once you note that the x-axis in Figure 1-3 is logarithmic—Flash is 20 times less expensive.

Open-weight models have their parameters publicly available, allowing for transparency, community improvement, and customization. Examples include Llama, Mistral, DeepSeek, Qwen, and Falcon. These models offer strong performance but generally lag behind frontier models, and they can be fine-tuned on proprietary data but require more expertise to host. However, there are hosted API services such as Together.ai, as well as fully managed API endpoints on the hyperscalers, that address this issue.

Locally hostable models are designed to run on consumer or enterprise hardware without requiring cloud connectivity. Examples include Llama 8B and Gemma 2B, hardware-optimized versions of which are available through NVIDIA NIM. This allows for complete privacy, with no data leaving your local devices and no ongoing API costs. However, these models have significantly reduced capabilities compared to cloud models. The demand for frontier models in "air-gapped" systems that are disconnected from the internet has led some proprietary model vendors to offer this service as well—for example, Gemini can be run on-premises in Google Distributed Cloud and OpenAI can be run on Azure's on-premises offerings.

Agentic AI

The class in the Pydantic API that invoked the Claude model was called `Agent`:

```
from pydantic_ai import Agent
agent = Agent('anthropic:claude-3-7-sonnet-latest',
      ...
```

What is an agent? In computer science, the term *agent* has long been used to describe software entities that act on behalf of users or other programs. When you invoke a foundational model, you specify a role, provide some context, and ask it to carry out some instruction. In the computer science sense, then, the LLM is acting as your agent.

For example, here's an example of creating an agent to manage inventory levels in a store (the full code is on GitHub):

```
agent = Agent(
  f"anthropic:{MODEL_ID}",
  system_prompt="You are an inventory manager who orders just i
  ...
)
```

Autonomy

In AI, an agent is also expected to be somewhat autonomous. Here, the LLM functions as the agent's brain, so you don't need to tell it how exactly to manage inventory levels beyond the *goal* (which is to order "just in time").

Suppose you have a list of items in your inventory, plus data on how well they've been selling and how long they'll take to deliver:

```
@dataclass
class InventoryItem:
    name: str
    quantity_on_hand: int
    weekly_quantity_sold_past_n_weeks: [int]
    weeks_to_deliver: int
```

```
items = [
    InventoryItem("itemA", 300, [50, 70, 80, 100], 2),
    InventoryItem("itemB", 100, [70, 80, 90, 70], 2),
    InventoryItem("itemC", 200, [80, 70, 90, 80], 1)
]
```

Provide the list of items to the agent and it will figure out which ones to reorder:

```
result = agent.run_sync(f"""
Identify which of these items need to be reordered this week.

**Items**
{items}
""")
```

The result will include this, in part:

> itemB
>
> **quantity_to_order=300** reason_to_reorder='Current stock (100) is insufficient to cover projected demand over delivery time. Based on recent weekly sales (70-90 units), we need to **order enough to cover the 2-week delivery period** plus maintain safety stock.’

Compare this to traditional programming, where you’d have to write code to explicitly manage inventory. Such *autonomy*—which means the ability to operate independently without constant human guidance or being explicitly programmed to do so—is the key differentiator between traditional software and AI agents.

Characteristics of Agents

Besides autonomy, agents are usually expected to have the following characteristics:

Goal orientation

Agents work toward specific objectives, rather than simply responding to input prompts. The goal of the inventory manager agent, which is to manage inventory just in time, was set in its system prompt.

Planning and reasoning

Notice that the inventory manager agent was able to plan the steps to determine how many items to reorder. It identified the range of recent weekly sales, projected the maximum sale forward by the delivery window, determined how many items would be required, and then calculated the number of items to reorder. None of this needed to be explicitly programmed.

Perception and action

Agents can gather the data they need (“perceive”) and act on their environment. You can usually give them this ability by enabling them to call external functions (such as searching the web, invoking calculators, and writing to databases) through Tool Calling (Pattern 21), which we’ll discuss in Chapter 7. With Tool Calling, you can go beyond explicitly providing the list of items to the inventory manager. Instead, the agent can retrieve the quantity on hand and weekly sales from backend databases. Also, instead of just telling you that item B needs to be reordered, it can invoke an API, perhaps even an API on the vendor’s site, to place the order.

Adaptability and learning

How do you know that ordering 300 items is correct? A human inventory manager would plug in assumptions of weekly sales for the next two weeks and validate that the store will not run out—and an agent can do the same thing. In Chapters 6 and 9 (respectively), you’ll see patterns such as Reflection (Pattern 18) and Self-Check (Pattern 31) that allow an agent to evaluate its output and self-correct.

At the time of writing, agentic behavior remains an aspirational goal for applications built on foundational models—nondeterminism, hallucinations, and various other failure modes pose challenges to building fully autonomous AI applications. Take nondeterminism, for example—each time, you might get a different list and quantity of items to reorder. (Try it!) Planning works in simple cases, but not in hard ones. In Chapter 5, you’ll see patterns such as Chain of Thought (Pattern 13) that improve the ability of an agent to do planning and reasoning.

Many of the design patterns in this book are ways to make your AI applications more agentic, or at least to push the boundaries of what you can build.

Fine-Grained Control

Foundational models give you fine-grained control over the generation process by allowing you to control sampling and beam search (both of which are discussed later in this chapter). The generation settings of LLMs provide powerful tools for controlling the balance between deterministic, high-quality outputs and creative, diverse responses. Understanding settings' mathematical underpinnings, from logits to sampling strategies, can help you control model behavior in simpler ways than with the design patterns covered in later chapters of this book.

Logits

Language models have hundreds of layers, but the very last layer predicts the next word to continue the text generation. They don't predict just one token, but instead, they provide a set of candidate tokens and the probability that each of those tokens will be the next one.

Logits are the raw, unnormalized outputs from a language model's final layer before they're converted into probabilities. Logits represent the model's assessment of how likely each token in its vocabulary is to be the next token in a sequence. Suppose, for illustration purposes, that there are five possible continuations to a sequence being generated and that the logits of each of the possible continuations are as shown on the left-hand side of Figure 1-4.

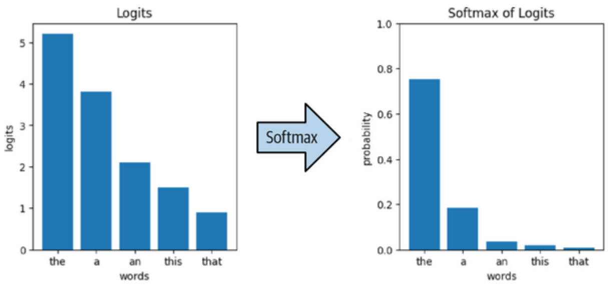


Figure 1-4. The softmax of the logits gives you the probability of various continuations

If the model were to use *greedy sampling*, where only the most likely word is chosen, it would simply select *the* since that word has the highest logits. However, such a strategy leads to highly repetitive and uninteresting text, so models use a sampling strategy in which all the possible continuation words have some nonzero probability of being selected. The transformation from logits to probabilities occurs through the `softmax` function:

$$P(token_i) = e^{logit_i} / \Sigma_j e^{logit_j}$$

Here, `P(token_i)` is the probability of selecting `token_i`. The `softmax` function accentuates the peaks and dampens the tails—for example, compare the length of the bars for *the* and *a* before and after the `softmax` function is applied in Figure 1-4.

If the distribution of potential continuations is less peaked, as shown in Figure 1-5, then the impact of the `softmax` is less pronounced.

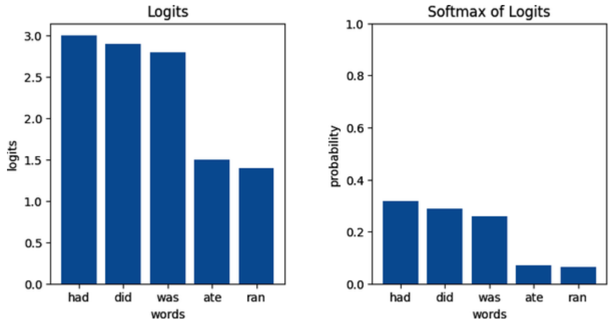


Figure 1-5. The impact of the `softmax` of the logits is less pronounced if the distribution is not peaked

We'll explore the use of logits to control style in Chapter 2.

Temperature

Temperature (T) is a hyperparameter that controls the randomness of token selection by scaling the logits before they're passed through the `softmax` function. The modified `softmax` equation with temperature is as follows:

$$P\left(token_i\right) = e^{logit_i/T} / \Sigma_j e^{logit_j/T}$$

The effect of scaling the same graphs with different values of T is shown in Figure 1-6.

As you can see, setting the temperature to zero turns on greedy sampling. As the temperature increases, the likelihood that less likely "tail" words will be chosen also increases. The effect of temperature is less pronounced when the distribution was less peaked to begin with.

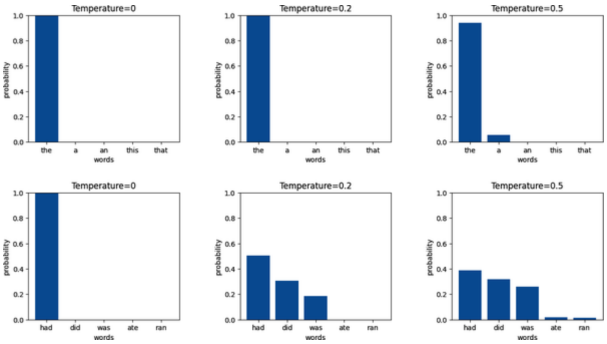


Figure 1-6. Effects of scaling the logits by temperature

Here's how to vary the temperature when using PydanticAI:

```python
agent = Agent('anthropic:claude-3-7-sonnet-latest',
              model_settings={
                 "temperature": 0.5
              },
              system_prompt="Complete the sentence.")
```

Here's how to do it when using the Anthropic API directly:

```python
completion = client.messages.create(
    model="claude-3-7-sonnet-latest",
    system="Complete the sentence.",
    temperature=0.5,
    messages=[
        ...
    ]
)
```

Here's an example, showing various continuations of the phrase *The trade war caused* produced at three different temperature settings:

> 0.0 : The trade war caused significant disruptions to global supply chains, leading to increased prices for consumers and economic uncertainty for businesses across multiple industries.

> 0.5 : The trade war caused significant disruptions to global supply chains, leading to increased prices for consumers and economic uncertainty for businesses across multiple industries. Many manufacturers were forced to reconsider their production strategies, while farmers faced reduced export opportunities as retaliatory tariffs limited access to international markets. The long-term effects included accelerated efforts to diversify supply chains away from affected regions and renewed debates about the effectiveness of protectionist trade policies.

> 0.8 : The trade war caused significant disruptions to global supply chains, forcing many companies to reconsider their manufacturing strategies and sourcing policies. It led to increased tariffs on imported goods, higher prices for consumers, and economic uncertainty in affected industries. Several businesses reported decreased profits as they absorbed additional costs or lost market share in foreign markets. The prolonged tension also contributed to volatility in financial markets and complicated diplomatic relations between the involved nations.

As you can see, increasing the temperature tends to lead to more creative output.

In Chapter 3 and Chapter 6, you will see situations, like RAG and LLM-as-Judge, where it may be necessary to use low or even zero temperatures.

Top-K Sampling

Top-K sampling restricts token selection to only the *k* most likely tokens from the vocabulary, effectively truncating the long tail of the probability distribution. This can help you avoid off-the-wall continuations at high temperatures.

Here's the impact of setting different top-K values to continue the phrase *The spaceship*:

> 1 : The spaceship zoomed through the vast expanse of space, its powerful engines glowing blue against the darkness as it carried its crew toward distant stars and unknown adventures.
>
> 10 : The spaceship glided silently through the vast emptiness of space, its powerful engines propelling it toward the distant galaxy where no human had ventured before.
>
> 100 : The spaceship soared through the starry expanse, its gleaming hull reflecting the distant light of alien suns as it carried its crew toward unexplored worlds beyond the edge of known space.

As you can see, when the top-K value is low, the generated text closely follows phrases that you can find in existing science fiction.

Nucleus Sampling

Nucleus sampling dynamically selects the smallest set of tokens whose cumulative probability exceeds a threshold *p*. Hence, it's also called *top-P sampling*. This creates a "nucleus" of tokens that represent the bulk of the probability mass. Figure 1-7 shows the impact of applying varying top-P values to our illustrative distributions.

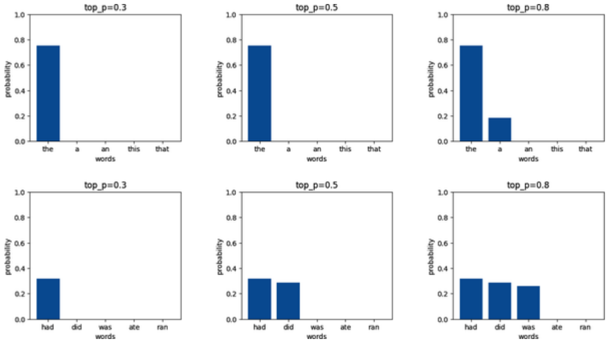


Figure 1-7. Effects of top-P sampling

Nucleus sampling adapts to the model's confidence at each step. When the model is very confident, few tokens are considered (as with low top-K), and when the model is uncertain, more tokens are considered (as with high top-K). Therefore, it generally produces more natural text than fixed top-K sampling does.

Beam Search

Models don't choose continuation tokens one at a time. Instead, they employ *beam search*, which is a deterministic search algorithm that aims to find the most likely sequence of tokens by exploring multiple possible continuations in parallel. In this way, the model can consider the probability of the entire sequence, not just the next token.

Some models give you knobs to control the beam search.

Repetition penalties reduce the probability of tokens that have already appeared in the generated text will appear again, thus helping to prevent repetitive loops. These penalties include the following:

Frequency penalty

> This increases the more a word or phrase appears in the generated text.

Presence penalty

This applies whenever a word or phrase is repeated, and it thus encourages the model to employ a diverse range of tokens in the generated text.

Length penalties adjust the scoring of sequences based on their length. These penalties include the following:

Minimum length penalty

This forces the model to generate at least a certain number of tokens.

Maximum length penalty

This caps generation at a specified length.

Length normalization penalty

This divides sequence scores by a function of their length to prevent bias toward shorter sequences.

Beam search width is the number of parallel sequences considered during generation. You will see beam search being used in patterns in Chapter 2 and Chapter 6.

At the time of writing, OpenAI and Gemini support repetition penalties but Anthropic does not. The Transformers library in Hugging Face supports length penalties and beam search width, but none of the hosted models do.

In-Context Learning

In "traditional" ML, where models are predominantly trained for single problems like classifications or regressions, models require extensive training to adapt their weights or parameters to a new task. But you can use LLMs for a wide variety of tasks by changing just the prompt without changing the weights.

Foundational models' capability to adapt to new tasks based solely on examples or instructions they're given in a prompt is called *in-context learning*. It emerges from training on diverse datasets that include various task formats and instructions. When given examples of a task within a prompt, the model identifies the underlying pattern and applies it to new instances. This process doesn't modify the model's weights, but rather, it leverages the model's existing knowledge to perform new tasks.

Zero-Shot Learning

Zero-shot learning involves instructing a model to perform a task without providing any examples. This technique relies on the model's pretrained knowledge and its ability to understand natural language instructions.

For example, you could ask Gemini-Flash to do the following:

> Analyze the use of light in Claude Monet's *Impression, Sunrise* and explain how it exemplifies impressionist techniques.

It would respond with something like this (though your results may vary, since model outputs are nondeterministic):

> In *Impression, Sunrise,* Monet employs light to capture a fleeting moment, prioritizing its transient effects over precise details. The hazy atmosphere and vibrant reflections on the water showcase the impressionist focus on perception and the subjective experience of light, rather than a realistic depiction of the scene. This emphasis on capturing the immediate "impression" of light became a hallmark of the Impressionist movement.

Few-Shot Learning

Few-shot learning extends this approach by including a small number of examples in the prompt. The examples serve as demonstrations that help the model understand the task's structure and expected output format, and the model then applies that structure to new instances. Because the examples are placed in the context of the prompt, this is also called *in-context learning*.

Few-shot learning is a simple and effective form of *context engineering*, which is exactly what it sounds like—adding finely crafted data and instructions to the context of the prompt to achieve the desired performance. You'll encounter many patterns in this book, especially in Chapter 5, that build on few-shot learning.

For example, suppose you want to build an application that allows the user to describe a painting they see in words and have the application identify it. You can teach it what goes into an identification by providing an example question and answer for *Impression, Sunrise* by Claude Monet:

~~~
agent = Agent(MODEL_ID,
              system_prompt="""You are an expert on art history
a painting. You should identify it.
""")

result = agent.run_sync("""
Example:
```
Description: shows two small rowboats in the foreground and a r
Answer:
   Painting: Impression, Sunrise
   Artist: Claude Monet
   Year: 1872
   Significance: Gave the Impressionist movement its name; capt
effects of light and atmosphere, with loose brushstrokes.
```

Description: The painting shows a group of people eating at a t
outside tent. The men are wearing boating hats.

""")
~~~

The result correctly identifies a Renoir painting that matches the description:

> Answer:
>
> Painting: Luncheon of the Boating Party
>
> Artist: Pierre-Auguste Renoir
>
> Year: 1881
>
> Significance: Captures a joyful social gathering of Renoir’s friends at the Maison Fournaise restaurant in Chatou, France, depicting the carefree atmosphere of Parisian life during the Belle Époque; known for its vibrant colors, lively composition, and portrayal of light and shadow.

Instead of requiring you to build a domain-specific model, collect lots of training data, and perform the training tasks, in-context learning allows LLMs to perform new tasks “on the fly.” The examples “guide” the LLM to mimic the input-output patterns they demonstrate.

When zero-shot learning (in which no examples are provided) does not solve your problem, you can try adding examples to your prompt and giving the model practical instructions to complete the task. Using in-context learning with a few examples is easier than using traditional ML because you don’t need to curate an extensive dataset to fine-tune a model. You can often achieve good results with few-shot learning.

With in-context learning, it’s possible to quickly prototype solutions. It also boosts production use cases: it’s much faster to update examples in a prompt than to curate, retrain, and redeploy an ML model.

However, in-context learning has a few limitations:

- It only works when the foundational model already has the necessary knowledge and capability.
- Adding many examples consumes valuable tokens of your model’s context window and will slow your inference time.
- LLMs sometimes struggle to generalize more complex problems based on a few examples.

In these scenarios, post-training might offer a better approach.

Post-Training

Post-training methods involve modifying the model weights of pretrained models to customize them to new tasks or domains. A post-trained model will have to be deployed and utilized from a different end point than the foundational model that was its starting point.

Post-Training Methods

Recall from Figure 1-2 that training a foundational model involves multiple stages. The first stage involves training a base LLM on predicting the next word, the second stage involves training the base LLM to perform tasks using SFT, and the third stage involves RLHF. You can post-train the model in any or all of these ways.

Here is some terminology you might encounter that is associated with post-training. These are not wholly different methods—there's some overlap between them:

Continued pretraining (CPT)

> You can continue training the base model if you have a dataset that contains vocabulary (such as industry jargon) and associations that the foundational model wasn't trained on. This requires access to the full weights and architecture of the base model. It's important to note that training base models is extremely expensive and time-consuming, and moreover, you'll have to perform the remaining stages of training on this new model. In March 2023, Bloomberg used this approach on financial documents, and just months later, it found that foundational models handily outperformed the domain-specific model. Since then, few organizations have chosen this approach.

Supervised fine-tuning (SFT)

> You can use SFT to further train language models on datasets consisting of (prompt, response) pairs in a supervised manner. If you use prompts that consist of specific instructions ("improve this management plan," etc.) and a wide variety of data, you can improve the model's ability to follow those specific natural language instructions. If you train the model on a wide variety of instructions, you can enhance its zero-shot capabilities across diverse tasks. You can also use such *instruction tuning* to enhance generalization or to elicit more helpful or honest behavior from the model. Whether the model will display this behavior will depend on the size and diversity of your dataset—if you do SFT on just a single task (instruction), then the model will probably forget the tasks it learned earlier and not generalize to new tasks. On the other hand, if your instruction training dataset consists of multiple tasks, then the model may generalize to new tasks even after just one round of SFT.

Parameter-efficient fine-tuning (PeFT)

> Because foundational models are so large, it's unwieldy to train them further. So, *parameter-efficient fine-tuning* (PeFT) approaches have emerged to make the training process more practical for large models. *Low-rank adaptation* (LoRA) represents weight updates using smaller matrices through low-rank decomposition. Instead of fine-tuning all model parameters, LoRA freezes the original pretrained weights and adds small, trainable "adapter" matrices that are decomposed into low-rank representations. LoRA drastically reduces the number of trainable parameters (making them up to 10,000 times fewer) and reduces GPU memory requirements (making them up to 3 times fewer). It also does not add inference latency, and it often performs on par with full fine-tuning. *Quantization-aware low-rank adaptation* (QLoRA) is an extension of LoRA that quantizes all the weights of the model. Its training process is more memory efficient, albeit slower. On the other hand, the quantized fine-tuned model takes up less space and is therefore faster than LoRA.

Preference tuning

> You can post-train a model by having it generate two outputs to the same prompt and giving it feedback on which output is better. When such preferences are provided by humans and the post-training is carried out through reinforcement learning, it's RLHF. Often, the post-training is carried out using direct preference optimization (DPO) because it is more efficient. DeepSeek introduced group relative policy optimization (GRPO), wherein multiple responses are generated and a response is assigned a score that is normalized by the average reward of the group.

The type of post-training you can do is intimately connected to the structure of the dataset. If the dataset consists purely of text completions, then you can use it to do unsupervised training, such as teaching a model new vocabulary or completely new associations via CPT. If the dataset consists of ideal responses to inputs (input-output pairs), then you can use it for SFT or instruction tuning. If the dataset consists of two responses to each input and notes which of the two is preferable, then you can use it for preference tuning. You can do any of these forms of post-training in a parameter efficient way or on quantized models.

You will encounter post-training in several patterns in this book, including Content Optimization (Pattern 5 in Chapter 2), Adapter Tuning (Pattern 15 in Chapter 5), and Prompt Optimization (Pattern 20 in Chapter 6). At the time of writing (June 2025), open weights models are the only ones that support all the forms of post-training above. If you are using a hosted model, please consult up-to-date documentation from your model provider to determine whether a pattern that requires post-training is possible (or change to a model that provides the needed capability).

Fine-Tuning a Frontier Model

Companies like OpenAI and Anthropic, as well as hyperscalers such as AWS and Google Cloud, have streamlined the process of post-training frontier models using SFT. It's possible to upload a training dataset of input-output pairs and launch the fine-tuning process, and the result will be an endpoint of an adapter-tuned model that can be used just like the foundational model.

We'll illustrate this by using OpenAI's GPT series of models, but fine-tuning Anthropic Claude on Amazon Bedrock or Google's Gemini on Vertex AI is quite similar. Once you generate your training pairs (you'll need at least a hundred pairs, but a couple of thousand pairs are even better), you'll need to store them in a JSON line-formatted file. You can then load the training dataset as follows:

```
training_file = client.files.create(
    file=open("training_data.jsonl", "rb"),
    purpose="fine-tune"
)
```

Once the data is loaded, you can kick off a fine-tuning job like this:

```
job = client.fine_tuning.jobs.create(
    training_file=training_file.id,
    model="gpt-3.5-turbo"  # Base model to fine-tune
)
```

The fine-tuning will take a few minutes to a few hours, depending on the size of the training dataset and the model. Once the training is completed, you can query the model identifier for your fine-tuned LLM like this:

```
job_status = client.fine_tuning.jobs.retrieve(job.id)
if job_status.status == 'succeeded':
    print(f"Model ID: {job_status.fine_tuned_model}")
```

The result will be a model ID of the following form:

```
ft:<BASE MODEL>-0125:<ORG_NAME>::<JOB ID>
```

Use this model ID in the inference API to invoke the fine-tuned model. It has the same API as the base model:

```
completion = client.chat.completions.create(
    model=job_status.fine_tuned_model,  # Use the fine-tuned m
    messages=messages
)
print(completion.choices[0].message.content)
```

Fine-Tuning an Open-Weight Model

Unsloth.ai provides you with the capability to fine-tune and train open-weight LLMs like Gemma and Llama. You can run Unsloth on your local hardware or use its managed fine-tuning services.

To fine-tune the 4-bit quantized version of Llama 3, start by loading in the model and its *tokenizer* (the class that will break an input text sequence into tokens of the sort the model expects):

```
from unsloth import FastLanguageModel
max_seq_length = 2048
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Meta-Llama-3.1-8B-bnb-4bit",
    max_seq_length=max_seq_length,
    load_in_4bit=True,
    dtype=None,
)
```

Then, attach a set of adapter weights to the base Llama model:

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=["q_proj", "k_proj", "v_proj", "up_proj", "d
                    "o_proj", "gate_proj"],
    use_gradient_checkpointing="unsloth"
)
```

In the previous code, you specified the *rank* (or *matrix size*) of the LoRA layer and the specific layers of the model to which they are to be applied. The previous code also applies LoRA to attention mechanisms (Q, K, and V matrices) and various projection layers.

Assuming that your input and output pairs are in one of the formats Unsloth supports, you can load the dataset using the following code:

```
dataset = load_dataset("...", split="train")
dataset = dataset.map(apply_template, batched=True)
```

Then, on a machine with sufficiently powerful GPUs, you can launch the training process by using the following code:

```
trainer=SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
)
trainer.train()
```

Once the model is trained, you can save it and push it to Hugging Face. There, you can merge the base model and its adapter layer into a single model before pushing it:

```
model.save_pretrained_merged("model", tokenizer, save_method="m
model.push_to_hub_merged("...", tokenizer, save_method="merged_
```

Then, you can use this model just like the base model.

Considerations

Fine-tuning models gives you the unique ability to customize LLMs to your domain-specific use cases, but fine-tuned models come with additional complexities—so make sure that the benefits of fine-tuning are worth these additional headaches:

Data requirements

Instead of providing a few in-context examples, you must use a more significant number of samples (more than a hundred) to customize the range of tasks the LLM can handle. This will require you to collect the samples ahead of time before attempting to fine-tune your LLMs. If you don't have the time or resources right now to do that, consider starting with in-context learning, collecting the data, and then fine-tuning your model later to boost its performance and provide more consistent, reliable outputs.

Catastrophic forgetting

Fine-tuning LLMs can lead to *catastrophic forgetting*, in which the model overemphasizes the examples provided during fine-tuning and loses its previously acquired knowledge. This wipes out LLMs' primary advantage:

their comprehensive world knowledge. You can mitigate this issue by fine-tuning your model on a small dataset, for only a few epochs, and by selecting an appropriate learning rate during the fine-tuning process. Generally, you should begin the fine-tuning learning rate at the value where the pre-training phase concluded (typically around 1e-5).

Additional complexity

Before releasing the fine-tuned model to a production environment, you need to evaluate its performance and make sure it hasn't picked up any counterproductive constructs, like biased language. Also, whenever a new version of your foundational model is released, you must fine-tune the model again and redo the training. Finally, you need to track the lineage of the training and validation data used to train the fine-tuned model. These are additional tasks that require careful execution, and using in-context learning is far simpler since it only requires you to add a handful of examples.

Additional costs

Providers like OpenAI charge a higher per-token rate for inference on fine-tuned models than for requests to their standard models. They charge a higher price because prompts to fine-tune models can be much shorter and can elicit the same or higher-quality output. Since providers calculate prices on input and output tokens, they increase token pricing to recover the overhead costs of hosting your fine-tuned model.

On the other hand, when you fine-tune open models, the model inference cost reduces—but you must pay for the GPU costs while performing the fine-tuning procedures. Depending on the base LLM, this can start at a few dollars but quickly spiral to hundreds of dollars per model version.

The Organization of the Rest of the Book

The rest of this book covers 32 design patterns, organized into eight chapters. You'll learn how to control model outputs, enhance knowledge retrieval, improve reasoning capabilities, increase reliability, enable action, optimize performance, and implement safeguards. The section on each pattern includes a clear problem statement, a solution approach, practical usage scenarios, and code examples. We hope that this book will help you learn how to build robust and effective GenAI applications.

In Chapter 2, we show you how to control the style and format of AI-generated content—which is a critical skill for ensuring brand consistency, accuracy, and compliance. You'll learn how to implement Logits Masking (Pattern 1) to ensure that text conforms to specific style rules by intercepting the generation process at the sampling stage. The section on Grammar (Pattern 2) will teach you to constrain outputs to specific formats or data schemas using formal grammar specifications. With Style Transfer (Pattern 3), you'll discover how to convert content to mimic specific tones through few-shot learning or fine-tuning. The section on Reverse Neutralization (Pattern 4) will show you how to generate content in specialized styles by first creating neutral content and then transforming it. Finally, Content Optimization (Pattern 5) will equip you with methods to determine optimal content styles through systematic comparison and preference tuning—which are particularly valuable for marketing, advertising, and educational materials where effective style factors aren't immediately obvious.

In Chapters 3 and 4, you'll learn patterns that can help you build AI systems that leverage external knowledge sources to address fundamental limitations like knowledge cutoffs, confidential data access, and hallucinations. You'll begin with Basic RAG (Pattern 6) and learn to ground AI responses in relevant information from knowledge bases. The section on Semantic Indexing (Pattern 7) will teach you to capture meaning across different media types by using embeddings, thus moving beyond simple keyword matching. With Indexing at Scale (Pattern 8), you'll master techniques for managing outdated or contradictory information through metadata, filtering, and reranking. Index-Aware Retrieval (Pattern 9) will equip you with advanced methods like hypothetical answers, query expansion, and GraphRAG to improve retrieval quality. Node Postprocessing (Pattern 10) will show you how to handle irrelevant content and ambiguous entities through reranking and contextual compression. You'll learn to build Trustworthy Generation (Pattern 11) systems that maintain user trust despite inevitable errors, and finally, the section on Deep Search (Pattern 12) will teach you iterative processes for complex information retrieval that overcome context window constraints and enable multihop reasoning.

In Chapter 5, we discuss powerful techniques to enhance the reasoning and specialized capabilities of language models. You'll learn Chain of Thought (CoT) (Pattern 13), which enables models to break down complex problems into intermediate reasoning steps and dramatically improve their performance on mathematical problems and logical deductions. The section on Tree of Thoughts (ToT) (Pattern 14) will teach you to implement tree search approaches for problems requiring exploration of multiple solution paths—which are ideal for strategic thinking and planning tasks. With Adapter Tuning (Pattern 15), you'll discover how to efficiently specialize large models by training small add-on neural network layers while keeping original model weights frozen, thus making specialized adaptation practical with limited data (from 100 to 10,000 examples). Finally, the section on Evol-Instruct (Pattern 16) will show you how to efficiently generate high-quality instruction-tuning datasets by evolving instructions through multiple iterations, thus enabling you to teach models new domain-specific tasks without extensive manual data creation.

In Chapter 6, you'll encounter patterns for building more reliable AI systems that can be trusted in production environments. You'll learn LLM-as-Judge (Pattern 17) to evaluate generative AI capabilities through detailed, multidimensional feedback—which is a foundational skill for comparing models and tracking improvements. The section on Reflection (Pattern 18) will teach you how to enable models to correct earlier responses based on feedback, which significantly improves reliability in complex tasks. With Dependency Injection (Pattern 19), you'll master techniques for independently developing and testing each component of an LLM chain, thus making your systems more maintainable and robust. Finally, the section on Prompt Optimization (Pattern 20) will show you how to systematically set and update prompts by optimizing them on example datasets, which reduces maintenance overhead when dependencies change and ensures consistent performance over time.

In Chapter 7, we discuss ways to transform your AI systems from passive information providers into active agents that can take meaningful actions in the world. You'll master Tool Calling (Pattern 21) to learn how to bridge LLMs with software APIs so they can invoke functions with appropriate parameters and incorporate the results into their responses. This enables real-time data access, connections to enterprise systems, and complex calculations. The section on Code Execution (Pattern 22) will teach you to leverage LLMs to generate code that can be executed by external systems—which is perfect for creating visualizations, annotating images, or updating databases. With Multiagent Collaboration (Pattern 23), you'll learn to design systems of specialized single-purpose agents that are organized in ways that mimic human organizational structures, thus enabling complex reasoning, multistep problem solving, collaborative content creation, and self-improving systems that can handle extended interactions without human intervention.

In Chapter 8, you'll learn essential patterns for deploying generative AI within real-world constraints of cost, latency, and computational resources. The section on Small Language Model (SLM) (Pattern 24) will demonstrate how to leverage smaller, more efficient models that can run on edge devices or with limited resources while still delivering acceptable performance for specific tasks. With Prompt Caching (Pattern 25), you'll discover techniques to reduce redundant computations and API calls and thus significantly lower costs for frequently requested content. Inference Optimization (Pattern 26) will equip you with methods to maximize throughput and minimize latency through techniques like speculative decoding, continuous batching, and prompt compression. The section on Degradation Testing (Pattern 27) will show you how to systematically evaluate model performance across different deployment scenarios and thus ensure consistent quality over time. Finally, the Long-Term Memory (Pattern 28) section will demonstrate how to maintain user history and dynamically apply personalization.

Chapter 9 will equip you with critical patterns for ensuring your generative AI applications operate safely, ethically, and within appropriate boundaries. The section on Template Generation (Pattern 29) will teach you how to pregenerate and review templates that require only deterministic string replacement at inference time—which is ideal for high-volume personalized communications where human review isn't scalable. With Assembled Reformat (Pattern 30), you'll learn to separate content creation into low-risk steps: first assembling data safely and then formatting it attractively to reduce the risk of inaccurate or hallucinated content. The section on Self-Check (Pattern 31) will show you how to use token probabilities to detect potential hallucinations cost-effectively in factual responses. Finally, Guardrails (Pattern 32) will equip you with comprehensive approaches to wrapping LLM calls with preprocessing and postprocessing layers that enforce security, privacy, content moderation, and alignment constraints—which is essential whenever your application could face malicious adversaries.

Finally, Chapter 10 demonstrates how the patterns from the first nine chapters can be composed into a production-ready agentic application.

---

[1] Look carefully at the sketch. Do you see any issues? Do all the characters have the appropriate number of fingers? Is the girl's face symmetric? Does the board game correspond to anything recognizable? At the time of writing, image models struggle with symmetry, count, and exact recall.

[2] A difference of 400 in the Elo rating corresponds to 10:1 odds that the higher-ranked player will beat the lower-ranked one.

Table of contents collapsed